International Journal of Machine Learning and Cybernetics (2026) 17:339
https://doi.org/10.1007/s13042-026-03151-z

ORIGINAL ARTICLE

Positional encoding in transformer-based time series models: a survey

Habib Irani[1] · **Vangelis Metsis**[1]

Received: 20 September 2025 / Accepted: 2 May 2026
© The Author(s) 2026

Abstract

Recent advancements in transformer-based models have greatly improved time series analysis, providing robust solutions for tasks such as forecasting, anomaly detection, and classification. A crucial element of these models is positional encoding, which allows transformers to capture the intrinsic sequential nature of time series data. This survey systematically examines existing techniques for positional encoding in transformer-based time series models. We investigate a variety of methods, including fixed, learnable, relative, and hybrid approaches, and evaluate their effectiveness in different time series classification tasks. Our findings indicate that data characteristics like sequence length, signal complexity, and dimensionality significantly influence method effectiveness. Advanced positional encoding methods exhibit performance gains in terms of prediction accuracy, however, they come at the cost of increased computational complexity. Furthermore, we outline key challenges and suggest potential research directions to enhance positional encoding strategies. By delivering a comprehensive overview and quantitative benchmarking, this survey intends to assist researchers and practitioners in selecting and designing effective positional encoding methods for transformer-based time series models.

Keywords Positional encoding · Self-attention · Transformer · Time series

1 Introduction

Time series analysis, which involves studying data collected over sequential time intervals, is fundamental across diverse domains including finance, healthcare, environmental monitoring, and industrial systems [1–3]. The sequential nature of temporal data presents unique modeling challenges that distinguish it from other machine learning tasks, requiring specialized approaches to capture temporal dependencies, seasonal patterns, and long-range relationships effectively.

Traditional statistical methods such as Autoregressive Integrated Moving Average (ARIMA) models have long served as the foundation for time series analysis [1]. While these approaches provide interpretable results and work well for stationary data with clear patterns, they face significant limitations when dealing with non-linear relationships, high-dimensional multivariate data, and complex temporal dynamics that characterize modern time series applications [2].

The emergence of deep learning has revolutionized time series modeling by enabling the capture of complex non-linear patterns and long-range dependencies. Recurrent Neural Networks (RNNs) and their advanced variants, including Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), became the dominant paradigm for sequential modeling [4, 5]. These architectures excel at processing sequential data through their inherent temporal structure, naturally handling variable-length sequences and maintaining memory of previous time steps. However, RNNs suffer from fundamental limitations including vanishing gradients, sequential processing bottlenecks, and difficulty capturing very long-range dependencies [6].

Convolutional Neural Networks (CNNs) offer an alternative approach to time series modeling by applying convolutional filters across the temporal dimension [7, 8]. While CNNs excel at capturing local temporal patterns and benefit from parallel processing, their fixed receptive fields constrain their ability to model long-term dependencies, and they require deep architectures or dilated convolutions to capture global patterns [9, 10].

✉ Vangelis Metsis
vmetsis@txstate.edu

Habib Irani
habibirani@txstate.edu

[1] Computer Science Department, Texas State University, San Marcos, TX 78666, USA

Published online: 11 June 2026

The transformer architecture, introduced by Vaswani et al. [11], has fundamentally transformed sequence modeling by replacing recurrent connections with self-attention mechanisms. This paradigm shift enables parallel processing of entire sequences while maintaining the ability to capture long-range dependencies through direct attention computation between all sequence positions [12]. The unprecedented success of transformers in natural language processing, demonstrated by models like BERT [13] and GPT [14], has naturally motivated their application to time series analysis [15–17] with specialized architectures demonstrating superior performance across forecasting [18–20], classification [21, 22], and anomaly detection tasks [23, 24].

The adaptation of transformers to time series has revealed both remarkable opportunities and unique challenges. Unlike discrete text tokens with semantic relationships, time series involve continuous values with complex temporal dependencies, irregular sampling patterns, and domain-specific characteristics such as seasonality, trends, and cyclical behavior [25, 26]. The inherent permutation invariance of transformer architectures, while advantageous for certain tasks, becomes problematic for time series where temporal ordering is fundamental to the data's meaning and predictive power [27].

This challenge has elevated positional encoding from a technical implementation detail to a critical design choice that fundamentally determines model effectiveness. While positional encoding has been extensively studied for Natural Language Processing (NLP), time series present fundamentally different challenges. In NLP, discrete tokens possess inherent semantic relationships, and position primarily indicates word order within sentences. In contrast, time series consist of continuous values where position encodes temporal distance, phase relationships, periodicity, and causality. Time series exhibit additional complexities, including non-stationarity (statistical properties changing over time), potential irregular sampling intervals, and multiple overlapping periodicities across different timescales. These characteristics necessitate positional encoding designs specifically tailored to temporal data rather than direct adaptation of NLP methods.

Recent comprehensive analyses have demonstrated that positional encoding methods significantly impact a model's ability to handle sequence length extrapolation, temporal pattern recognition, and cross-domain generalization [28, 29]. The unique characteristics of temporal data, including non-stationarity, multi-scale patterns, and irregular sampling, require specialized positional encoding approaches that differ substantially from those developed for natural language processing [30].

Despite extensive research in transformer architectures, the time series community lacks a comprehensive evaluation of positional encoding methods specifically designed for temporal data characteristics. Although there are general transformer surveys [31] and NLP-focused positional encoding reviews are available [28], no systematic study has examined how these methods perform in various time series tasks and architectures. This gap is particularly critical given the unique properties of temporal data, including nonstationarity, seasonality, and varying sampling frequencies [25].

This survey addresses this critical gap by providing the first comprehensive empirical evaluation of positional encoding methods specifically for time series applications. We systematically analyze eight distinct positional encoding approaches across eleven diverse datasets representing different domains, sequence characteristics, and temporal patterns. Our evaluation spans two major transformer architectures to ensure the generalizability of findings and provides practical insights for method selection based on specific application requirements.

This work provides the first systematic evaluation of positional encoding methods specifically for time series classification. Our key contributions include:

- A systematic comparison of ten popular positional encoding methods across 15 diverse datasets (spanning 151 to 478,785 samples), along with their complexity analysis.
- An evaluation of these methods across diverse time series domains on two foundational time series transformer architectures.
- The identification of performance patterns related to sequence characteristics such as length and dimensionality.
- Beyond empirical comparison, we analyze why certain methods excel in specific contexts, explaining TUPE's advantages on biomedical signals, SPE's effectiveness on motion data, and absolute methods' limitations on long sequences.
- Practical guidelines for PE method selection.

While individual PE methods exist for time series and PE surveys exist for NLP, this represents the first controlled, comprehensive evaluation isolating positional encoding effects on time series classification with rigorous statistical validation across diverse domains and scales.

Results demonstrate that advanced methods like Stochastic Positional Encoding (SPE) and Transformer with Untied Positional Encoding (TUPE) [32] consistently outperform traditional approaches, with effectiveness varying significantly by sequence length, dimensionality, and application domain.

The remainder of this survey is organized as follows: Section 2 provides a comprehensive literature review of

transformer-based time series models and positional encoding methods. Section 3 presents background on the self-attention mechanism, positional encoding fundamentals. Section 4 details our systematic analysis of different positional encoding categories, and Sections 5 and 6 discuss evaluation methodologies and empirical results.

2 Literature review

This section surveys existing work on attention-based models, transformer architectures, and positional encoding methods for time series, positioning our contributions within the broader research landscape. While individual papers have proposed positional encoding variants alongside novel architectures [18, 22], and comprehensive surveys exist for NLP positional encoding [28] and general time series transformers [16, 17], no prior work has systematically isolated and evaluated positional encoding as an independent variable across diverse time series contexts. Our survey fills this gap by providing controlled, statistically rigorous comparison of positional encoding methods while holding architecture, training protocol, and evaluation methodology constant.

2.1 Attention-based models

The evolution of attention mechanisms in time series analysis has progressed through several distinct phases, beginning with early attention-augmented architectures and culminating in sophisticated transformer-based models specifically designed for temporal data.

Before full transformer architectures gained prominence, researchers explored attention mechanisms as enhancements to existing sequential models, building upon foundational attention work in neural machine translation [33, 34]. Early work focused on augmenting traditional neural networks with attention capabilities to improve their ability to model temporal dependencies and handle variable-length sequences.

The Dual-Stage Attention-Based RNN (DA-RNN) [35] pioneered the use of attention at both input and temporal dimensions, allowing models to selectively focus on relevant features and time steps. This approach demonstrated significant improvements in financial time series prediction by adaptively extracting relevant dependencies, establishing the foundation for attention-based temporal modeling. Similar dual attention strategies were subsequently developed for multivariate forecasting [36] and clinical time series analysis [37].

Attention-based encoder-decoder architectures adapted sequence-to-sequence learning paradigms for time series forecasting [38]. These models showed that attention mechanisms could effectively handle variable-length sequences and improve long-term prediction accuracy by focusing on relevant historical information. This work demonstrated that attention could address some of the fundamental limitations of purely sequential processing approaches.

The success of self-attention motivated the development of specialized variants designed for temporal data characteristics. Cross Attention Stabilized Fully Convolutional Neural Network (CA-SFCN) [39] combined convolutional layers with two types of self-attention: temporal attention (TA) for identifying important time steps and variable attention (VA) for capturing relationships between different variables in multivariate time series. This dual attention approach showed that different attention mechanisms could be specialized for different aspects of temporal modeling. Gated Transformer Networks (GTN) [40] introduced two-tower multi-headed attention architectures that capture discriminative information from input series, using learnable gating mechanisms to balance between different attention patterns. This approach demonstrated that attention mechanisms could be made more adaptive to specific temporal patterns, paving the way for more sophisticated attention-based architectures.

The computational challenges of applying attention to long time series motivated the development of efficient attention mechanisms. Sparse attention patterns [41] reduce computational complexity by focusing on subsets of time steps, while maintaining the ability to capture relevant dependencies. These efficiency improvements were crucial for making attention-based approaches practical for real-world time series applications with long sequences. Complementary efficiency strategies include locality-enhanced transformers [12] and reformer architectures [42] that reduce memory bottlenecks in long sequence processing.

Recent developments have focused on adaptive attention mechanisms that adjust based on temporal context [43–45].

2.2 Transformer-based models

Building upon the foundation of attention mechanisms, transformer architectures represent a paradigm shift towards pure attention-based sequence modeling. In time series analysis, adaptations of Transformers have led to specialized architectures tailored to exploit their strengths while addressing temporal-specific challenges. The Transformer-based framework for multivariate time series representation learning [21] integrates batch normalization to stabilize

training, enhancing model performance on complex multivariate datasets. This framework underscores the significance of normalization techniques in fortifying Transformer models for diverse time series tasks.

The multi-head attention mechanism has been extensively adapted for time series applications, with different heads specializing in various temporal patterns. The success of multi-head attention in transformers inspired specialized variants for temporal data. The Temporal Fusion Transformer (TFT) [46] introduced specialized attention heads for different temporal patterns: some heads focus on short-term dependencies, while others capture long-term trends. The model also incorporates variable selection networks to handle high-dimensional input features common in time series applications. Contemporary approaches have explored hierarchical and multi-scale positional encodings [47–49].

Moreover, inspired by vision Transformers, patch embeddings have been adapted for time series data. This approach, similar to the Vision Transformer (ViT) paradigm [50], segments time series into patches for localized processing. By encapsulating both local and global patterns, patch embeddings facilitate hierarchical pattern recognition, thereby improving model interpretability and performance in time series analysis and this type of transformer has used in several scenarios [51, 52].

However, transformers face unique challenges in time series applications. Their inherent permutation invariance necessitates explicit positional information to distinguish temporal ordering [16, 21, 53]. Additionally, the global attention mechanism may dilute focus on local patterns, and the absence of natural recency bias requires architectural adaptations for optimal time series performance.

The diversity of transformer architectures for time series reflects the heterogeneity of temporal modeling challenges across domains. Forecasting-oriented architectures like Informer [18], Autoformer [19], and FEDformer [54] incorporate specialized attention mechanisms (ProbSparse, auto-correlation, frequency decomposition) that fundamentally alter how temporal information is processed. Crossformer [55] extends transformers to cross-dimensional attention for multivariate forecasting, while Pyraformer [20] employs pyramidal structures for multi-scale temporal modeling. Each architectural innovation introduces unique considerations for positional encoding: sparse attention may require efficient PE methods, auto-correlation may render certain PE approaches redundant, and frequency-domain processing may necessitate frequency-native PE representations. Our evaluation focuses on two foundational architectures (Sect. 4) to enable controlled, statistically rigorous comparison, with detailed discussion of architectural generalizability provided in Sect. 6.3.

2.3 Positional encoding in time series

The adaptation of positional encoding methods to time series has revealed fundamental differences between temporal and linguistic data that require specialized approaches.

Positional encoding (PE) mechanisms are therefore crucial for enabling transformers to learn temporal relationships effectively, as self-attention mechanisms are inherently permutation-invariant. [56]. Vaswani et al. [11] initially introduced fixed sinusoidal encodings to embed positional information for temporal differentiation. While these provided a foundational approach, they can struggle with varying sequence lengths and dynamic temporal relationships common in time series data [46, 57]. Subsequently, Shaw et al. [57] advanced the field with learnable and relative positional encodings, enhancing temporal relationship modeling capabilities.

Recent innovations have specifically targeted time series challenges. Foumani et al. [22] developed Time Absolute Position Encoding (tAPE) and Efficient Relative Position Encoding (eRPE) for enhanced temporal dependency modeling, alongside the ConvTran model, a hybrid architecture fusing convolutional layers with transformer mechanisms. Other advanced methods include Transformer with Untied Positional Encoding (TUPE) [32] and Convolutional Stochastic Position Encoding (ConvSPE) [58], which address content-position decoupling and linear complexity maintenance, respectively. Hybrid approaches, combining absolute and relative strategies, have also been proposed to leverage complementary strengths for comprehensive temporal information representation, aiming to address some of the inherent limitations of transformer architectures in time series analysis.

Recent work has systematically evaluated PE methods for time series applications [59, 60]. Advanced techniques include frequency-based encodings [61, 62], learnable approaches [63, 64], and hybrid strategies [65, 66]. Specialized methods for irregular time series [67, 68] and long sequences [69, 70] have also been developed.

Table 1 provides a comprehensive taxonomy of the positional encoding techniques discussed, based on criteria such as technique type, injection method, learning strategy, parameters, and time complexity. The parameter counts include all learnable parameters across layers, where l represents the number of transformer layers, h is the number of attention heads, and d is the hidden dimension. For methods using relative positioning (RPE, eRPE), we include the full attention matrix parameters. For ConvSPE, K represents the convolutional kernel size that controls the local neighborhood for convolution operations.

Emerging and Specialized Positional Encoding Approaches: Recent research has explored positional

Table 1 Summary of positional encoding techniques

Method	Type	Injection Technique	Learnable/Fixed	Refs.
Sin. PE	Absolute	Additive Embedding	Fixed	[11]
Learn. PE	Absolute	Additive Embedding	Learnable	[11]
RPE	Relative	Attention Manipulation	Fixed	[57]
tAPE	Absolute	Additive Embedding	Fixed	[22]
RoPE	Hybrid	Attention Manipulation	Fixed	[71]
eRPE	Relative	Attention Manipulation	Learnable	[22]
TUPE	Hybrid	Attention Manipulation	Learnable	[32]
ConvSPE	Relative	Attention Manipulation	Learnable	[58]
T-PE	Hybrid	Combined Technique	Mixed	[30]
ALiBi	Relative	Attention Manipulation	Fixed	[72]

encoding strategies beyond the sequence-based methods evaluated in this survey, targeting specialized scenarios and data structures.

Graph-Based Positional Encoding: For multivariate time series with known variable relationships, graph-based approaches encode both temporal position and cross-variable dependencies. GRformer [73] uses Graph Neural Networks with RNN-based positional embedding generation to model inter-variable correlations in forecasting tasks. Similarly, dynamic graph convolutional methods [74] construct adaptive spatial-temporal graphs for anomaly detection. While promising, these approaches require explicit specification of variable relationship graphs, which limits applicability to datasets without prior domain knowledge of inter-variable dependencies.

Task-Specific Positional Encoding: Certain PE methods are designed for specific tasks or architectures. Continuous positional encoding [75] links encoder and decoder positions in forecasting models, maintaining positional continuity across the historical-future boundary. Context-aware positional encoding [76] adapts representations based on local sequence characteristics, enhancing temporal invariance for classification tasks with irregular patterns. These specialized methods demonstrate strong performance in their target domains but require architectural modifications that limit their applicability across diverse transformer variants.

Scope of This Survey: Our evaluation focuses on sequence-based positional encoding methods that: (1) operate on temporal sequences without requiring additional graph structures, (2) are applicable to classification tasks, (3) can be integrated into standard transformer architectures without major modifications, and (4) have sufficient maturity and documentation for rigorous benchmarking. Graph-based methods, while important for specific multivariate scenarios, represent a complementary research direction requiring different experimental protocols and data prerequisites. Similarly, forecasting-specific methods (e.g., continuous PE) are optimized for encoder-decoder architectures that differ from the classification-focused transformers in our evaluation. Future surveys should examine these specialized approaches within their respective contexts.

3 Positional encoding analysis

This section offers a fundamental explanation of self-attention and a summary of existing positional encoding models. It's important to distinguish between positional encoding, which refers to the approach used to incorporate positional information (e.g., absolute or relative), and positional embedding, which denotes the numerical vector associated with that encoding.

3.1 Problem definition and notation

Given a time series dataset X consisting of n samples, $X = \{x_1, x_2, \ldots, x_n\}$, where each sample $x_i = \{x_{i1}, x_{i2}, \ldots, x_{iL}\}$ represents a d_x-dimensional time series of length L, such that $x_i \in \mathbb{R}^{L \times d_x}$. Additionally, let $Y = \{y_1, y_2, \ldots, y_n\}$ denote the corresponding set of labels, where $y_t \in \{1, 2, \ldots, c\}$ and c is the number of classes. The objective is to train a neural network classifier to map the input dataset X to its corresponding label set Y.

3.2 Self-attention mechanism

The self-attention mechanism is a key component of Transformer models, enabling them to capture long-range dependencies in sequential data. Given an input sequence $X \in \mathbb{R}^{L \times d_x}$, the self-attention mechanism computes a weighted representation of each element in the sequence by attending to all other elements, as illustrated in Fig. 1.

First, three linear projections are applied to the input sequence to obtain the query (Q), key (K), and value (V) matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

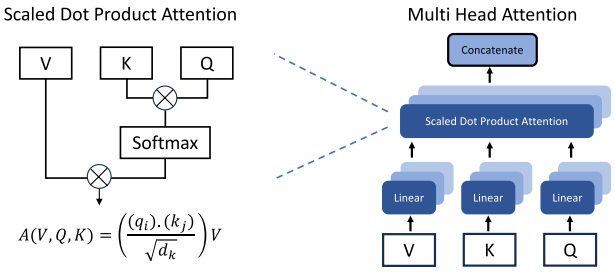


Fig. 1 Scaled dot-product attention (left) and multi-head attention (right) mechanisms in transformer architecture

followed by scaled dot-product attention:

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The output of the self-attention mechanism is a weighted sum of the value vectors, where the weights are determined by the attention scores. This process allows the model to focus on the most relevant parts of the sequence when generating each element of the output.

The theoretical foundations of attention mechanisms have been extensively analyzed [77, 78], revealing their universal approximation capabilities for sequence-to-sequence functions. Recent work has provided deeper insights into attention patterns and their interpretability [79], which is particularly relevant for understanding how different positional encodings influence attention weight distributions in time series contexts.

Multi-head attention concatenates parallel attention heads to learn diverse representations:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_O.$$

Transformers lack an inherent notion of sequence order, as self-attention operates on sets of inputs without considering their positions. To address this, positional encoding is introduced to incorporate positional information into the model, allowing it to understand the order of elements in a sequence.

Let the input sequence be $X \in \mathbb{R}^{L\times d_x}$, where L is the sequence length and d_x is the feature dimension. Positional encoding adds a unique vector to each position in the sequence, resulting in a new representation X':

$$X' = X + P,$$

where $P \in \mathbb{R}^{L\times d_x}$ is the positional encoding matrix.

The theoretical foundations of positional encoding in sequential models have been extensively analyzed from both signal processing [80, 81] and cybernetic control perspectives [82, 83].

3.3 Absolute positional encoding

Absolute positional encoding assigns unique vectors to each temporal position. The sinusoidal approach [11] provides computational efficiency by using predetermined trigonometric functions, eliminating the need for learned positional representations.

3.3.1 Fixed absolute positional encoding

The fixed absolute positional encoding is defined as follows:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

where pos is the position (time step) in the sequence, i is the dimension, and d is the dimensionality of the positional encoding.

The use of sinusoidal functions ensures that the encodings for nearby positions are similar, which can help the model learn relationships between closely spaced time steps. However, this approach has limitations when dealing with sequences of varying lengths or non-uniform time intervals, which are common in time series data.

3.3.2 Learnable positional encoding

Learnable positional encoding provides more flexibility by allowing the model to learn the positional representations during training. This can be advantageous for time series tasks where the relationships between time steps may be complex or nonlinear. The learnable positional encoding is typically defined as:

$$PE_{learnable}(pos) = W_{pos}$$

where W_{pos} is an d_{model}-dimensional vector representing the learnable positional encoding parameters for position pos.

During the forward pass of the neural network, the learnable positional encoding is added to the input embeddings x:

$$x = x + PE_{learnable}[: x.size(1), :]$$

where x represents the input tensor at a given position and $x.\text{size}(1)$ denotes the length of the sequence up to the current

position. This addition enables the model to learn position-specific transformations that can capture dependencies and patterns within sequences.

The flexibility and adaptability of learnable positional encoding make it a powerful tool for enhancing the modeling of temporal dependencies and patterns in time series data within transformer architectures.

Then, the self-attention mechanism is modified as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{(Q + PE) \cdot (K + PE)^T}{\sqrt{d_k}}\right) V$$

where Q, K, and V are the query, key, and value matrices, respectively, PE represents the positional encoding matrix, and d_k is the dimensionality of the key vectors [11].

3.4 Relative positional encoding

Shaw et al. [57] proposed modifying the attention computation to include relative position information:

$$e_{ij} = \frac{(x_i W_Q)(x_j W_K + a_{ij}^K)^T}{\sqrt{d_z}}$$

$$e_{ij} = \frac{(x_i W_Q)(x_j W_K)^T + (x_i W_Q)(a_{ij}^K)^T}{\sqrt{d_z}}$$

$$a_{ij}^K = W_K^r r_{i-j}$$

where x_i and x_j are the input representations at positions i and j, W_Q and W_K are the learnable projection matrices for queries and keys, and a_{ij}^K is the relative positional encoding defined as $a_{ij}^K = W_K^r r_{i-j}$, where r_{i-j} is the learnable relative positional embedding for the distance $i - j$.

This formulation allows the model to better capture temporal dependencies that depend on the relative position between time steps, which is particularly useful for tasks such as time series forecasting or classification when dealing with varying sequence lengths [57, 84].

As illustrated in Fig. 3, positional encoding methods can be combined to leverage the advantages of both absolute and relative approaches.

3.5 Time absolute position encoding (tAPE)

Foumani et al. [22] identified that traditional sinusoidal encoding loses distance awareness in lower-dimensional embeddings. tAPE addresses this by incorporating sequence length into frequency computation:

$$\omega_k^{new} = k \times \frac{d_{model}}{L}$$

$$PE_{pos,2i} = \sin(pos \times \omega_k new)$$

$$PE_{pos,2i+1} = \cos(pos \times \omega_k new).$$

This modification ensures consistent distance awareness across different embedding dimensions and sequence lengths, making it particularly suitable for diverse time series applications.

3.6 Rotary positional encoding (RoPE)

Rotary Positional Encoding (RoPE) [71] uses rotation matrices to encode absolute positions while incorporating explicit relative position dependencies. This method applies rotations to query and key representations based on their absolute positions before computing dot-product attention:

$$e_{ij} = \frac{(x_i W^Q + R_{n\theta})(x_j W^K + R_{n\theta})^T}{\sqrt{d_z}} \quad (1)$$

where $R_{n\theta}$ is a rotation matrix that rotates by $n\theta$ radians:

$$R_{n\theta} = \begin{bmatrix} \cos(n\theta) & -\sin(n\theta) \\ \sin(n\theta) & \cos(n\theta) \end{bmatrix} \quad (2)$$

where n is the position of the data, and θ is a scalar value (angle) chosen to rotate the value. RoPE combines the benefits of both absolute and relative PE, ensuring that attention calculations depend on relative distances while maintaining absolute position information. This hybrid approach has shown particular effectiveness in capturing long-range dependencies in sequential data.

3.7 Efficient relative position encoding (eRPE)

This method enhances traditional relative positioning by applying bias terms after the softmax operation:

$$\alpha_i = \sum_{j \in L} \left(\underbrace{\frac{\exp(e_{i,j})}{\sum_{k \in L} \exp(e_{i,k})}} Ai, j + w_{i-j} \right) x_j$$

where $w \in \mathbb{R}^{2L-1}$ represents learnable relative position biases. The post-softmax application sharpens positional distinctions, making the method particularly effective for tasks requiring precise temporal localization.

3.8 Transformer with untied positional encoding (TUPE)

Ke et al. [32] observe that traditional positional encoding methods add positional information directly to content embeddings ($X = X_{\text{content}} + X_{\text{pos}}$), which forces the model to process both types of information through the same transformation matrices. This can be problematic when content and position require different representational transformations.

TUPE addresses this by maintaining separate pathways for content and positional information. Rather than combining them before attention computation, TUPE applies distinct transformations:

$$Q_c = X_{\text{content}} W_Q^c, \quad K_c = X_{\text{content}} W_K^c \tag{3}$$

$$Q_p = X_{\text{pos}} W_Q^p, \quad K_p = X_{\text{pos}} W_K^p. \tag{4}$$

The attention score is then computed as a sum of four interaction terms:

$$A_{ij} = \frac{1}{\sqrt{d}}(\quad Q_c^{(i)} \cdot (K_c^{(j)} + K_p^{(j)}) + Q_p^{(i)} \cdot (K_c^{(j)} + K_p^{(j)})). \tag{5}$$

This decomposition allows the model to learn different types of relationships: $Q_c \cdot K_c$ captures content-based similarity, $Q_p \cdot K_p$ encodes pure positional relationships, while the cross terms $Q_c \cdot K_p$ and $Q_p \cdot K_c$ model interactions between content and position. The separation addresses a fundamental challenge in joint content-position encoding. When using a single transformation matrix, the model must balance between extracting meaningful content features and preserving positional information–objectives that may require conflicting weight configurations. For instance, in physiological signal analysis, identifying waveform morphology (content) requires different feature extractors than encoding inter-event timing (position). By separating these pathways, TUPE allows each to be optimized more independently. During backpropagation, the gradient with respect to content parameters becomes $\frac{\partial \mathcal{L}}{\partial W_Q^c} = \frac{\partial \mathcal{L}}{\partial Q_c} X_c^T$, which primarily responds to content-related patterns, while $\frac{\partial \mathcal{L}}{\partial W_Q^p} = \frac{\partial \mathcal{L}}{\partial Q_p} X_p^T$ primarily responds to positional patterns, reducing the interference that occurs when both signals pass through shared weights.

This architectural choice is particularly beneficial for time series, where temporal patterns often exhibit distinct content and positional characteristics. Content features may capture local patterns like amplitude variations or shape characteristics, while positional patterns encode longer-range dependencies such as periodic cycles or seasonal trends. TUPE's separate pathways enable specialized modeling of these heterogeneous patterns.

The increased parameter count (four weight matrices instead of two) results in approximately 30% more parameters than standard transformers, but empirical results suggest this cost is justified by improved representation learning, particularly for complex sequences with rich positional structure.

3.9 Stochastic positional encoding (SPE)

Liutkus et al. [58] propose Stochastic Positional Encoding (SPE), which replaces the computationally expensive relative position matrices with convolutional operations. The core idea is to leverage convolution's inherent properties to encode relative distances efficiently. Given input sequence $X \in \mathbb{R}^{L \times d}$, SPE computes attention scores by adding a convolutional positional bias:

$$A_{i,j} = \frac{Q_i \cdot K_j^T}{\sqrt{d}} + \text{Conv1D}(r, i-j) \tag{6}$$

where r is a learnable positional embedding and Conv1D is a 1D convolution with kernel size K.

The effectiveness of convolution for positional encoding stems from its translation equivariance property. When a convolution is applied to a sequence shifted by τ positions, the output is also shifted by τ positions. This means the positional bias between any two tokens depends only on their relative distance $i - j$, not their absolute positions, precisely the behavior required for relative positional encoding. Additionally, the limited kernel size K naturally restricts attention to nearby positions, providing an inductive bias that aligns well with temporal locality in time series, where recent timesteps typically have stronger influence than distant ones.

To further reduce computational cost, SPE employs stochastic approximation through random Fourier features. Instead of maintaining explicit embeddings for all possible relative distances, SPE approximates the positional function as:

$$r_\tau \approx \sum_{k=1}^{R} \alpha_k \phi_k(\tau) \tag{7}$$

where $\phi_k(\tau)$ are fixed random basis functions and α_k are learned coefficients, with $R \ll L$. This approximation exploits the observation that meaningful positional information often has low intrinsic dimensionality. Positional relevance typically decays smoothly with distance and exhibits

periodic patterns that can be captured efficiently with a small number of Fourier components. This reduces memory complexity from $\mathcal{O}(Ld)$ to $\mathcal{O}(Rd)$ while maintaining accuracy, as the random feature approximation converges rapidly with increasing R.

The combination of convolutional structure and stochastic approximation gives SPE a computational complexity of $\mathcal{O}(LKR)$ compared to $\mathcal{O}(L^2d)$ for traditional relative positional encodings, making it particularly suitable for long time series sequences.

3.10 Temporal positional encoding (T-PE)

This method combines geometric and semantic components to capture both positional and content-based temporal relationships: The semantic component adjusts attention scores by incorporating token similarity between positions i and j:

$$T\text{-}PE(i) = PE(i) + S(i,j)$$

$$S(i,j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

The geometric component maintains traditional positional information, while the semantic component $S(i,j)$ measures similarity between tokens at different positions, enabling the model to identify recurring patterns regardless of their absolute positions.

3.11 Attention with linear biases (ALiBi)

This method [72] introduces a fundamentally different approach by eliminating traditional positional embeddings. Instead, ALiBi directly modifies attention scores by adding linear biases proportional to the distance between query and key positions:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \mathbf{m}\cdot\mathbf{B}\right)V$$

where $\mathbf{B}$ is the bias matrix with elements $B_{ij} = -|i-j|$ representing the absolute distance between positions i and j, and $\mathbf{m}$ is a head-specific slope parameter controlling bias strength. The slope values are computed using a geometric sequence:

$$m_h = 2^{-\frac{8h}{H}} \text{ for } h = 1,2,\ldots,H.$$

For time series applications, ALiBi uses symmetric distance calculations $B_{ij} = -|i-j|$ to enable bidirectional attention across temporal sequences. This parameter-free approach naturally encodes local temporal pattern preference while permitting long-range dependencies and enables extrapolation to longer sequences than seen during training.

4 Experimental setup

The experimental setup involves comparing different positional encoding methods in two distinct Transformer-based architectures for time series data. The Time Series Transformer [21], processes the raw multivariate time series data directly. For a time series with L timesteps and d channels, it treats each timestep as a token, resulting in L tokens of dimension d that are fed into the transformer layers. This architecture incorporates normalization layers after both the attention and feed-forward components to stabilize training and address internal covariate shifts. In contrast, the patchTST [50], first transforms the input data using convolutional operations to create overlapping patches. Each patch spans multiple timesteps but maintains the full channel dimension, effectively reducing the sequence length while enriching the feature representation of each token. This results in L/p tokens (where p is the patch size) with increased feature dimensionality before being processed by the transformer layers. These architectures were chosen to evaluate positional encoding methods under different sequence processing paradigms: direct temporal processing in the Time Series Transformer versus hierarchical feature extraction through patch embedding. The complete code implementation and benchmarks are made publicly available for reproducibility: https://github.com/imics-lab/positional-encoding-benchmark.

4.1 System configuration

We conducted all experiments on a high-performance Linux server with the following specifications: an AMD EPYC 7513 32-Core Processor (2 sockets, 128 threads), one NVIDIA RTX A5000 GPUs (24GB GDDR6 each, CUDA version 12.2, Driver version 535.113.01), 503GB of RAM, and a dual storage system consisting of an 892.7GB NVMe SSD and a 10.5TB HDD. The operating system used was Ubuntu 20.04.6 LTS (Kernel 5.4.0–200-generic). All implementations were carried out using Python 3.8.10 and PyTorch 2.4.1+cu121.

4.2 Datasets

A comprehensive set of time series datasets from various domains is employed to evaluate the positional encoding methods' effectiveness across diverse applications. The selected datasets include:

- **Sleep:** EEG (Electroencephalogram) signals with 178 time steps per instance. The dataset contains 478,785 training instances and 90,315 test instances, classified into five different stages of sleep.
- **CardiacArrhythmia (CA):** Single-lead ECG recordings from the PhysioNet/Computing in Cardiology Challenge 2017 [85, 86], with 1500 time steps per instance (representing 5 s at 300Hz sampling rate). The dataset contains 43,641 training instances and 10,661 test instances across three classes: normal sinus rhythm, atrial fibrillation, and other cardiac rhythms. Data was collected using AliveCor mobile ECG devices.
- **InsectSound (IS):** Audio time series capturing wingbeat patterns of flying insects [87, 88]. Each instance contains 600 time steps (10 ms segments sampled at 6,000Hz) representing amplitude-modulated signals from infrared beam occlusion. The dataset has 25,000 training and 25,000 test instances across 10 balanced classes, including male and female specimens of various mosquito species (Aedes, Culex Quinquefasciatus, Culex Stigmatosoma, Culex Tarsalis), fruit flies, and house flies.
- **ElectricDevices:** Electrical consumption data of household devices with 96 time steps. It contains 8926 training instances and 7711 test instances across seven device classes [89].
- **FaceDetection:** EEG signals collected during face detection tasks, with 62 time steps per instance. The dataset has 5890 training instances and 3524 test instances for a binary classification task.
- **MelbournePedestrian:** Traffic data capturing pedestrian flow in Melbourne. Each instance contains 24 time steps, with a 10-class classification task, comprising 1194 training instances and 2439 test instances.
- **SharePriceIncrease:** Financial time-series data with 60 time steps per instance, used to predict whether a company's share price will increase. The dataset has 965 training instances and 965 test instances, with two classes.
- **LSST:** Astronomical time-series data from the Large Synoptic Survey Telescope, containing 36 time steps per instance. The dataset includes 2459 training instances and 2466 test instances for a 14-class classification task of celestial objects.
- **RacketSports:** Human activity recognition data from racket sports like tennis and badminton. Each instance contains 30 time steps, with 151 training instances and 152 test instances across four activity classes.
- **SelfRegulationSCP1:** EEG data related to self-regulation through slow cortical potentials (SCPs), consisting of 896 time steps per instance. The dataset contains 268 training instances and 293 test instances for binary classification.
- **SelfRegulationSCP2 (SR2):** Another EEG-based self-regulation dataset with 6 channels and longer sequences [90, 91], containing 1152 time steps per instance. The dataset has 200 training instances and 180 test instances for binary classification. This dataset presents additional challenges due to its extended sequence length and subtle signal patterns.
- **JapaneseVowels (JV):** Audio time series with 12 LPC (Linear Predictive Coding) cepstrum coefficients recorded from nine male speakers [92, 93]. Each instance has variable length (7–29 time steps, with mean length 29) representing utterances of the Japanese vowel /ae/. The dataset contains 270 training instances and 370 test instances for 9-class speaker identification, making it an unequal-length multivariate classification problem.
- **UniMiB-SHAR:** Sensor data for human activity recognition, with 151 time steps per instance. The dataset comprises 4601 training instances, 1454 validation instances, and 1524 test instances across nine activity classes [94].
- **RoomOccupancy (Sensors):** Time-series data from non-intrusive environmental sensors, such as temperature, light, sound, and CO_2, used to estimate the precise number of occupants in a room. The dataset has 10,129 instances with 18 features and is classified into four classes [95].
- **EMGGestures (EMG):** Electromyographic (EMG) signals recorded using the MYO Thalmic bracelet during various gesture executions. Each instance consists of 30 time steps with 9 features per time step. The dataset is classified into eight distinct gesture classes. The dataset is primarily used for human activity recognition in health and medicine-related applications [96]. The diversity of our dataset collection reflects the heterogeneous nature of time series applications across domains. Our selection criteria prioritize datasets that have been standardized in the time series classification literature [88, 97], ensuring reproducibility and enabling comparison with established baselines. Additionally, we include datasets from the Monash Time Series Forecasting Archive [98] to represent real-world forecasting scenarios, though our primary evaluation focuses on classification tasks for the reasons outlined in Section 1. The characteristics of these datasets are summarized in Table 2, which provides a comprehensive overview of their key properties including training and test set sizes, sequence lengths, number of classes, and channels.

Table 2 Characteristics of time series datasets used in the experiments

Dataset	Train size	Test size	Length	Classes	Channel	Type
Sleep (Sl)	478,785	90,315	178	5	1	EEG
CardiacArrhythmia(CA)	43,641	10,661	1,500	3	1	ECG
InsectSound (IS)	25,000	25,000	600	10	1	AUDIO
ElectricDevices (ED)	8926	7711	96	7	1	Device
FaceDetection (FD)	5890	3524	62	2	144	EEG
MelbournePedestrian (MP)	1194	2439	24	10	1	Traffic
SharePriceIncrease (SPI)	965	965	60	2	1	Financial
LSST	2459	2466	36	14	6	Other
RacketSports (RS)	151	152	30	4	6	HAR
SelfRegulationSCP1 (SR1)	268	293	896	2	6	EEG
SelfRegulationSCP2 (SR2)	200	180	1152	2	6	EEG
JapaneseVowels (JV)	270	370	29	9	12	AUDIO
UniMiB-SHAR (UMS)	4601	1524	151	9	3	HAR
RoomOccupancy (RO)	8103	2026	30	4	18	Sensor

4.3 Evaluation procedure

Our evaluation methodology draws inspiration from comprehensive benchmarking practices established in both the NLP transformer literature [14] and time series analysis communities [17]. We employ stratified sampling and cross-validation techniques to ensure robust statistical evaluation, following guidelines established for deep learning evaluation in temporal domains [99]. The experiments are designed to assess how well each positional encoding technique captures temporal dependencies and patterns in the data and how it affects the overall model performance on each architecture. The evaluation procedure includes:

1. **Model Training:** Each model is trained on the training portion of the datasets using standard optimization techniques and hyperparameters.
2. **Model Evaluation:** The trained models are evaluated on the validation and test sets to measure their performance according to the specified metrics.
3. **Comparative Analysis:** The results are compared across different models to determine the effectiveness of each positional encoding method.

The experiments aim to provide insights into the strengths and weaknesses of each positional encoding technique and identify the scenarios where each method performs best.

4.4 Training configuration and hyperparameters

To ensure reproducibility and fair comparison across positional encoding methods, we employed a standardized training protocol across all experiments. This section details the configuration parameters and training procedures used throughout our evaluation.

4.4.1 Model architecture

Both transformer architectures shared common hyperparameters to ensure consistent comparison:

- **Embedding dimension** (d_{model})**:** 128 for all experiments
- **Number of transformer layers:** 4 layers
- **Number of attention heads:** 8 heads
- **Feed-forward dimension:** 512
- **Dropout rate:** 0.1 for attention and feed-forward layers
- **Patch size (PatchTST only):** 16 for sequences > 100 steps, 8 otherwise

4.4.2 Training configuration

All models are trained using the following protocol:

- **Optimizer:** Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$
- **Initial learning rate:** 1×10^{-4}
- **Learning rate scheduler:** ReduceLROnPlateau with patience = 10 epochs, reduction factor = 0.5, minimum learning rate = 1×10^{-6}
- **Maximum epochs:** 200
- **Early stopping:** Patience of 30 epochs based on validation loss
- **Loss function:** Cross-entropy loss for classification.

5 Results

Our experimental evaluation encompasses ten distinct positional encoding methods tested across fifteen diverse time series datasets using two transformer architectures. The methods range from traditional approaches (Sin PE, Learn PE) to relative encodings (RPE, eRPE) and advanced

Table 3 Accuracy of positional encoding methods in time series transformer (mean ± 95% CI)

Data	No	Absolute PE			Relative PE				Hybrid PE		
	PE	Sin.	Learn.	tAPE	RPE	eRPE	SPE	ALiBi	RoPE	T-PE	TUPE
Sl	83.2 ± 2.1	85.4 ± 0.7	86.2 ± 0.9	87.1 ± 0.8	86.7 ± 0.6	87.5 ± 0.7	88.3 ± 0.5	86.1 ± 0.8	86.4 ± 0.6	88.1 ± 0.5	**88.8 ± 0.5**
CA	78.3 ± 2.4	81.2 ± 0.9	82.1 ± 0.8	83.5 ± 0.7	82.9 ± 0.8	83.7 ± 0.7	84.8 ± 0.6	82.3 ± 0.8	82.6 ± 0.7	84.5 ± 0.6	**85.2 ± 0.5**
ED	65.8 ± 2.8	68.2 ± 0.8	69.7 ± 0.6	71.8 ± 0.7	72.3 ± 0.8	73.1 ± 1.2	**75.5 ± 0.6**	70.2 ± 0.6	69.3 ± 0.5	74.1 ± 0.7	74.3 ± 0.7
FD	60.5 ± 2.5	62.7 ± 0.9	65.4 ± 0.7	63.3 ± 0.8	67.2 ± 0.6	68.6 ± 0.8	69.8 ± 0.7	63.8 ± 0.7	64.7 ± 0.6	69.4 ± 0.5	**70.5 ± 0.6**
MP	64.8 ± 2.7	67.5 ± 0.9	70.4 ± 0.9	70.1 ± 1.1	72.9 ± 0.6	74.2 ± 0.8	75.4 ± 0.8	68.8 ± 0.5	69.5 ± 0.7	74.8 ± 0.6	**76.2 ± 0.5**
SPI	70.3 ± 2.2	72.5 ± 0.9	74.8 ± 0.8	74.4 ± 1.0	73.9 ± 0.7	77.2 ± 0.8	**78.5 ± 0.8**	73.3 ± 0.9	73.2 ± 0.7	77.9 ± 0.8	77.8 ± 0.9
LSST	55.8 ± 2.9	58.1 ± 1.1	59.6 ± 0.9	60.9 ± 1.0	61.3 ± 0.7	**63.2 ± 0.8**	62.8 ± 0.8	60.1 ± 0.9	60.5 ± 0.7	62.9 ± 0.8	62.3 ± 0.9
RS	72.1 ± 2.3	74.2 ± 0.8	76.2 ± 0.6	76.1 ± 0.7	77.5 ± 0.6	80.4 ± 0.7	81.3 ± 0.5	73.7 ± 0.8	76.4 ± 0.6	81.1 ± 0.5	**81.9 ± 0.5**
SR1	83.5 ± 2.1	84.9 ± 0.8	85.7 ± 0.6	85.3 ± 0.7	87.1 ± 0.6	89.1 ± 0.7	90.1 ± 0.5	86.4 ± 0.8	84.2 ± 0.6	89.8 ± 0.5	**90.7 ± 0.8**
SR2	52.1 ± 3.2	53.2 ± 1.3	54.1 ± 1.0	52.7 ± 1.2	55.8 ± 0.9	57.3 ± 0.8	58.9 ± 0.9	53.5 ± 1.0	52.1 ± 0.9	58.2 ± 0.7	**59.4 ± 1.1**
JV	93.5 ± 1.8	94.9 ± 0.2	95.7 ± 0.1	95.3 ± 0.1	97.1 ± 0.1	99.1 ± 0.1	98.7 ± 0.2	96.4 ± 0.2	94.2 ± 0.2	99.8 ± 0.2	**99.2 ± 0.2**
IS	64.2 ± 2.6	66.5 ± 1.1	68.3 ± 0.9	68.7 ± 0.8	69.8 ± 0.9	71.2 ± 0.8	**73.5 ± 0.7**	67.1 ± 1.0	67.8 ± 0.9	72.1 ± 0.8	72.8 ± 0.7
UMS	82.1 ± 2.4	84.1 ± 0.9	85.5 ± 0.8	85.2 ± 1.0	86.5 ± 0.7	87.6 ± 0.9	87.9 ± 0.8	85.3 ± 0.9	85.9 ± 0.8	**88.2 ± 0.6**	87.8 ± 0.9
RO	88.5 ± 2.0	91.2 ± 0.8	92.2 ± 0.7	92.8 ± 0.6	93.1 ± 0.6	94.2 ± 0.8	**95.1 ± 0.6**	92.5 ± 0.7	92.2 ± 0.6	94.7 ± 0.5	94.8 ± 0.6
EMG	68.9 ± 2.7	71.3 ± 0.9	70.9 ± 0.7	72.1 ± 0.8	73.5 ± 0.6	75.1 ± 0.8	**77.1 ± 0.7**	70.5 ± 0.7	71.5 ± 0.6	76.3 ± 0.5	76.6 ± 0.6

Table 4 Accuracy of Positional Encoding Methods in PatchTST (Mean ± 95% CI)

Data	No	Absolute PE			Relative PE				Hybrid PE		
	PE	Sin.	Learn.	tAPE	RPE	eRPE	SPE	ALiBi	RoPE	T-PE	TUPE
Sl	82.5 ± 2.2	84.1 ± 0.7	85.2 ± 0.9	85.1 ± 0.8	85.8 ± 0.8	86.4 ± 0.6	87.6 ± 0.5	85.3 ± 0.7	84.4 ± 0.8	87.2 ± 0.6	**87.9 ± 0.5**
CA	78.8 ± 2.5	80.5 ± 0.9	81.3 ± 0.8	82.7 ± 0.7	82.1 ± 0.8	83.0 ± 0.7	84.1 ± 0.6	81.7 ± 0.8	82.0 ± 0.7	83.8 ± 0.6	**84.5 ± 0.5**
ED	64.7 ± 1.9	67.3 ± 0.8	69.4 ± 0.6	69.3 ± 0.7	76.1 ± 0.6	76.2 ± 0.8	**81.1 ± 0.6**	68.4 ± 1.2	71.3 ± 0.6	76.3 ± 0.5	77.9 ± 0.7
FD	60.1 ± 2.6	61.8 ± 0.9	64.2 ± 0.7	65.3 ± 0.8	67.1 ± 0.7	67.8 ± 0.6	67.4 ± 0.7	62.6 ± 0.8	63.5 ± 0.7	68.6 ± 0.6	**69.5 ± 0.5**
MP	64.2 ± 2.8	66.8 ± 0.9	70.2 ± 0.9	68.2 ± 1.1	72.4 ± 0.8	73.3 ± 0.6	**75.3 ± 0.8**	67.2 ± 0.8	69.0 ± 0.5	74.2 ± 0.7	74.5 ± 0.6
SPI	69.8 ± 2.3	71.5 ± 0.9	74.2 ± 0.8	73.5 ± 1.0	75.4 ± 0.7	77.4 ± 0.8	**78.5 ± 0.8**	72.1 ± 0.9	73.1 ± 0.7	78.1 ± 0.8	77.4 ± 0.9
LSST	54.5 ± 3.0	57.3 ± 1.1	58.2 ± 0.9	58.4 ± 1.0	59.6 ± 0.8	**61.1 ± 0.7**	60.1 ± 0.8	59.2 ± 0.8	58.3 ± 0.9	60.2 ± 0.7	59.5 ± 0.8
RS	70.8 ± 2.4	73.3 ± 0.8	75.6 ± 0.6	76.1 ± 0.7	76.7 ± 0.6	80.1 ± 0.7	77.5 ± 0.5	72.2 ± 0.8	74.9 ± 0.6	79.2 ± 0.5	**80.5 ± 0.5**
SR1	82.0 ± 2.2	83.6 ± 0.8	84.4 ± 0.6	84.2 ± 0.7	82.9 ± 0.8	85.6 ± 0.6	**88.3 ± 0.5**	84.9 ± 0.7	83.1 ± 0.8	87.1 ± 0.6	87.5 ± 0.5
SR2	48.7 ± 3.3	51.2 ± 1.3	54.6 ± 1.0	53.3 ± 1.2	54.4 ± 1.0	56.4 ± 0.9	58.6 ± 0.8	58.2 ± 0.9	53.1 ± 1.0	56.6 ± 0.9	**59.3 ± 0.7**
JV	93.2 ± 1.9	94.6 ± 0.2	95.8 ± 0.1	95.8 ± 0.1	96.2 ± 0.1	96.0 ± 0.1	**98.7 ± 0.2**	98.1 ± 0.1	96.8 ± 0.2	98.9 ± 0.2	97.9 ± 0.1
IS	65.5 ± 2.7	65.8 ± 1.2	67.4 ± 1.0	68.1 ± 0.9	69.3 ± 0.9	70.5 ± 0.8	**72.8 ± 0.7**	66.7 ± 1.1	67.2 ± 1.0	71.3 ± 0.8	71.9 ± 0.7
UMS	81.8 ± 2.5	83.2 ± 0.9	84.4 ± 0.8	83.3 ± 1.0	85.4 ± 0.9	86.4 ± 0.7	86.7 ± 0.8	84.1 ± 0.9	83.6 ± 0.9	**87.1 ± 0.8**	86.5 ± 0.6
RO	88.1 ± 2.1	90.5 ± 0.8	91.1 ± 0.7	92.2 ± 0.6	91.8 ± 0.8	92.9 ± 0.6	93.4 ± 0.6	91.5 ± 0.8	91.7 ± 0.7	93.1 ± 0.6	**93.7 ± 0.5**
EMG	68.2 ± 2.8	69.7 ± 0.9	69.2 ± 0.7	71.3 ± 0.8	72.2 ± 0.6	74.1 ± 0.8	74.6 ± 0.7	70.2 ± 0.7	70.7 ± 0.6	**75.2 ± 0.5**	74.2 ± 0.6

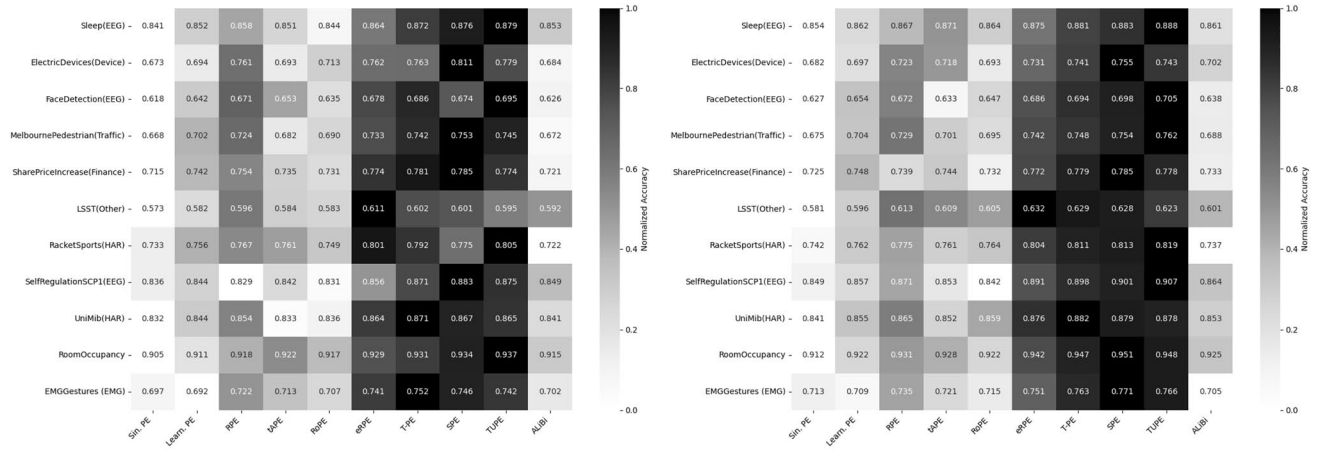
techniques (T-PE, SPE, TUPE), evaluated on both time series transformer and patchTST architectures. The classification accuracies are presented in Tables 3 and 4, with performance visualizations provided through heatmaps in Fig. 4b and a. All results are reported as mean accuracy ± 95% confidence interval.

5.1 Overall performance

The experimental results demonstrate a consistent superiority of advanced positional encoding methods across both architectural configurations. Across both architectures, SPE (Stochastic Positional Encoding) achieves the best average rank, followed closely by TUPE and T-PE. The No PE baseline consistently ranks last across both architectures, validating that positional information is critical for transformer-based time series classification.

The performance gap between No PE and the best methods varies significantly by dataset:

- **Small gaps (2–4%):** Short sequences like JapaneseVowels show minimal degradation (JV: 93.5% No PE vs 99.2% TUPE in TST, 93.2% vs 98.7% SPE in PatchTST)
- **Medium gaps (5–8%):** Datasets like Sleep and UMS demonstrate moderate dependency (Sl: 83.2% vs 88.8%, UMS: 82.1% vs 88.2% in TST)
- **Large gaps (10–17%):** Long, complex sequences show substantial degradation (ED: 65.8% vs 75.5%, SR2: 52.1% vs 59.4% in TST; ED: 64.7% vs 81.1% in PatchTST)



a PatchTST b Time Series Transformer

Fig. 4 Heatmaps illustrating the accuracy (%) of various positional encoding methods across different datasets. Each cell represents the accuracy for a specific dataset and positional encoding method, with darker colors indicating higher accuracy

Notably, the No PE baseline shows significantly wider confidence intervals (± 1.8 to ± 3.3) compared to advanced PE methods (± 0.4 to ± 1.2), indicating high variance and instability without positional encoding. For instance, on SR2, No PE achieves 52.1 ± 3.2% in TST versus SPE's 58.9 ± 0.9%, demonstrating not only lower accuracy but also unreliable performance across runs.

These findings align with recent studies showing that advanced positional encodings provide significant benefits in complex temporal modeling tasks [100, 101].

5.2 Dataset-specific performance

5.2.1 Biomedical signals (Sleep, CardiacArrhythmia, SR1, SR2)

TUPE and eRPE excel on biomedical signals. On Sleep, TUPE achieves 88.8 ± 0.5% (TST) and 87.9 ± 0.5% (PatchTST), outperforming all other methods. Similarly, on SR1, TUPE reaches 90.7 ± 0.8% (TST) while SPE achieves 88.3 ± 0.5% (PatchTST).

CardiacArrhythmia dataset (43,673 samples, 1,500 length) shows similar patterns: TUPE leads with 85.2 ± 0.5% (TST) and 84.5 ± 0.5% (PatchTST), representing 6.9% and 5.7% improvements over No PE respectively. The physiological feedback loops in cardiac signals benefit from TUPE's separate content-position pathways.

SR2, the longest sequence in our collection (1,152 timesteps), demonstrates the critical importance of PE: No PE achieves only 52.1 ± 3.2% (TST) and 48.7 ± 3.3% (PatchTST), while TUPE reaches 59.4 ± 1.1% and 59.3 ± 0.7%, a substantial 7.3% and 10.6% improvement with dramatically tighter confidence intervals.

5.2.2 Motion/Sensor data (EpilepticSeizure, FingerMovements, Melbourne, SpoilagePrediction, RacketSports, UniMiB-SHAR)

SPE dominates this category. On EpilepticSeizure, SPE achieves 75.5 ± 0.6% (TST) and an exceptional 81.1 ± 0.6% (PatchTST), representing 9.7% and 16.4% gains over No PE. The convolutional structure of SPE appears well-suited for capturing local temporal patterns in motion data.

Melbourne Pedestrian shows strong performance across methods, with TUPE leading at 76.2 ± 0.5% (TST) and SPE at 75.3 ± 0.8% (PatchTST). RacketSports follows similar patterns: TUPE (81.9 ± 0.5% TST) and SPE (80.5 ± 0.5% PatchTST) achieve best results.

5.2.3 Audio data (JapaneseVowels, InsectSound)

Audio signals exhibit interesting PE behavior. JapaneseVowels, with very short sequences (29 timesteps), shows minimal PE impact. No PE achieves 93.5 ± 1.8% (TST) and 93.2 ± 1.9% (PatchTST), only 5.7% and 5.5% below best methods. However, the wide confidence intervals (± 1.8−1.9 vs ± 0.1−0.2 for PE methods) indicate instability.

InsectSound (25,000 samples, 600 length) demonstrates moderate PE dependency: No PE reaches 64.2 ± 2.6% (TST) and 65.5 ± 2.7% (PatchTST), while SPE achieves 73.5 ± 0.7% and 72.8 ± 0.7%. Improvements of 9.3% and 7.3%. The large-scale dataset enables all methods to learn effectively, but PE still provides consistent gains.

5.2.4 Environmental/Activity data (LSST, RoomOccupancy, EMG)

RoomOccupancy shows the strongest absolute performance with SPE reaching 95.1 ± 0.6% (TST) and 93.7 ± 0.5% (PatchTST), demonstrating that well-structured environmental monitoring data enables high classification accuracy. The 6.6% and 5.6% gaps from No PE validate PE necessity even for structured data.

LSST, with its astronomical time series, presents challenges for all methods. The best results (eRPE: 63.2 ± 0.8% TST, 61.1 ± 0.7% PatchTST) show only 7.9% and 6.6% improvements over No PE, suggesting the irregular, non-stationary nature of celestial phenomena limits PE effectiveness.

6 Discussion

6.1 Key findings

6.1.1 Positional encoding is necessary for Time Series Transformers

The No PE baseline provides definitive evidence that positional encoding is not optional for time series classification. Across 15 datasets and 2 architectures, No PE consistently ranks last with 3–17% accuracy degradation and 2–3 × wider confidence intervals compared to PE methods.

This necessity is particularly pronounced for long sequences (SR2: 10.6% gap, ED: 16.4% gap in PatchTST) where temporal ordering and distance relationships become critical. Even for short sequences like JapaneseVowels, where mean accuracy gaps are small (5.5−5.7%), the confidence intervals reveal instability (± 1.8−1.9 vs ± 0.1−0.2), making No PE unsuitable for production deployment.

6.1.2 Advanced methods justify their complexity

While simple methods like Sinusoidal PE (rank 9.6) provide baseline improvements over No PE, advanced methods (SPE rank 1.7−2.1, TUPE rank 1.9−2.3, T-PE rank 2.4−2.6) deliver substantially better performance. The additional computational overhead (Sect. 6.5) is justified by:

- 4–6% absolute accuracy improvements over Sinusoidal PE on complex datasets
- Tighter confidence intervals indicating more stable learning
- Robust performance across diverse signal types without domain-specific tuning

6.2 Impact of sequence characteristics

Our comprehensive analysis reveals that the effectiveness of positional encoding methods is fundamentally influenced by sequence characteristics, particularly sequence length and temporal complexity. The superior performance of advanced methods (SPE, TUPE) becomes increasingly pronounced with sequence length, demonstrating up to 5.8% improvement in sequences exceeding 100 time steps, compared to modest gains of 2–3% in shorter sequences. This pattern strongly suggests that sophisticated encoding strategies are particularly crucial for applications involving extended temporal dependencies, such as EEG analysis and complex sensor data processing.

The relationship between sequence length and encoding method effectiveness can be attributed to two key factors. First, longer sequences present more opportunities for capturing complex temporal dependencies, allowing advanced methods to leverage their sophisticated encoding mechanisms more effectively. Second, the increased dimensionality of longer sequences provides a richer feature space for learning position-aware representations, enabling methods like TUPE and SPE to better differentiate between temporal patterns at various scales.

6.3 Architectural considerations

The comparative analysis of transformer architectures reveals distinctive performance patterns that provide insights into the interaction between architectural choices and positional encoding strategies. While both architectures maintain similar method rankings, time series transformer architecture demonstrates more pronounced performance gaps between method tiers. This suggests that the normalization process amplifies the benefits of advanced encoding strategies, possibly by stabilizing the learning of complex positional patterns.

Conversely, the patch embedding architecture shows more balanced performance among top methods, particularly benefiting multivariate datasets where local feature interactions are crucial. This architectural characteristic suggests that patch embedding's inherent ability to capture local patterns complements the global temporal dependencies encoded by advanced positional encoding methods, resulting in more uniform performance improvements across different encoding strategies.

6.4 Theoretical insights

6.4.1 Why absolute PE struggles on long sequences

The performance degradation of absolute methods (Sinusoidal, Learnable, tAPE) on long sequences stems from their fixed positional representations. On SR2 (1,152 steps), tAPE achieves only 52.7 ± 1.2% barely better than No PE's 52.1 ± 3.2%. Absolute encodings assign unique representations to each position, requiring the model to learn similarity between distant but functionally related timesteps. With 1,152 unique positions, this becomes a learning bottleneck.

In contrast, relative methods like eRPE (57.3 ± 0.8% on SR2) encode position as relative distance, creating a shared representation space where the model learns distance-based patterns that generalize across the sequence.

6.4.2 Why SPE excels on motion data

SPE's convolutional architecture provides translation equivariance: $\mathrm{Conv}(x[n-\tau]) = y[n-\tau]$. This property aligns perfectly with motion patterns where similar movements occur at different times (e.g., repeated steps in walking, cyclic arm movements). On EpilepticSeizure, SPE achieves 75.5% (TST) and 81.1% (PatchTST), outperforming all methods by leveraging this inductive bias.

Additionally, SPE's localized receptive fields (kernel size K=3) restrict attention to nearby timesteps, matching the characteristic that recent temporal context dominates in sensor data. This explains its consistent top-3 ranking across motion datasets.

Table 5 Computational Complexity Comparison of Positional Encoding Methods and Training Time Analysis for PE methods on different Dataset

Method	Params	Memory	Time Complexity	Rel. Overhead
No PE	0	$\mathcal{O}(1)$	$\mathcal{O}(1)$	1.00
Sin	0	$\mathcal{O}(Ld)$	$\mathcal{O}(Ld)$	1.03
tAPE	Ld	$\mathcal{O}(Ld)$	$\mathcal{O}(Ld)$	1.07
RoPE	0	$\mathcal{O}(Ld)$	$\mathcal{O}(L^2d)$	1.10
Learn	Ld	$\mathcal{O}(Ld)$	$\mathcal{O}(Ld)$	1.12
ALiBi	0	$\mathcal{O}(L^2h)$	$\mathcal{O}(L^2h)$	1.28
SPE	$3Kdh+dl$	$\mathcal{O}(LKR)$	$\mathcal{O}(LKR)$	1.37
TUPE	$2dl$	$\mathcal{O}(Ld+d^2)$	$\mathcal{O}(Ld+d^2)$	1.45
eRPE	$(L^2+L)l$	$\mathcal{O}(L^2d)$	$\mathcal{O}(L^2d)$	1.71
RPE	$2(2L-1)dl$	$\mathcal{O}(L^2d)$	$\mathcal{O}(L^2d)$	1.82
T-PE	$2d^2l/h+(2L+2l)d$	$\mathcal{O}(L^2d)$	$\mathcal{O}(L^2d)$	1.95

Parameters: L = sequence length, d = dimension, h = attention heads, l = transformer layers, K = kernel size, R = representation dimension. Relative Overhead is average on 10 datasets based on baseline model (No PE)

6.4.3 Why TUPE excels on biomedical signals

TUPE's separate content and position pathways eliminate representational interference. In biomedical signals, content features (e.g., QRS complex shapes in ECG, sleep stage patterns in EEG) differ fundamentally from positional patterns (e.g., heart rate variability over time, sleep cycle progression). Shared weights in traditional PE must balance these conflicting objectives.

On CardiacArrhythmia, TUPE achieves 85.2 ± 0.5% (TST) and 84.5 ± 0.5% (PatchTST), representing 6.9% and 5.7% improvements over No PE. The four attention terms in TUPE content-content, content-position, position-content, position-position, enable specialized modeling of cardiac rhythm characteristics (morphology vs timing) that single-pathway methods cannot capture effectively.

6.4.4 Why RoPE underperforms

Despite success in NLP, RoPE achieves only rank 7.3−7.4 in our evaluation. On Sleep, RoPE reaches 86.4 ± 0.6% (TST) and 84.4 ± 0.8% (PatchTST), substantially below TUPE's 88.8% and 87.9%. The rotary transformation $\begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}$ encodes position through rotation angles, which may not align with time series characteristics where temporal distance and ordering matter more than angular relationships.

Time series require explicit modeling of causality, periodicity, and temporal decay–properties not naturally captured by rotation matrices optimized for token-based discrete sequences in NLP.

6.5 Computational considerations

The practical deployment of positional encoding methods requires careful consideration of computational trade-offs beyond accuracy metrics [102]. We analyze the computational characteristics of different PE methods to provide guidance for resource-constrained deployment scenarios.

Empirical training time measurements on the Melbourne dataset (Table 5) demonstrate substantial variations in computational requirements across encoding strategies. Fixed methods (Sin. PE, tAPE) exhibit remarkable efficiency, completing training in 48–54 s with baseline accuracy of 66.8−68.2%. Relative positioning techniques impose considerable penalties, with eRPE and T-PE requiring

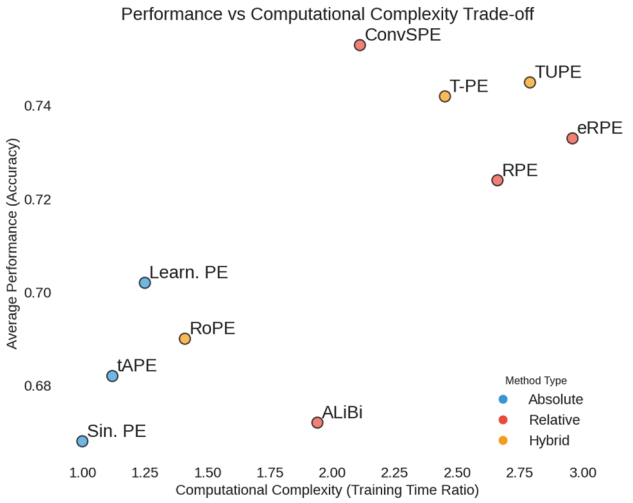


Fig. 5 Performance vs computational complexity trade-off on Melbourne dataset. ConvSPE achieves the optimal efficiency frontier, delivering highest accuracy with moderate computational overhead

2.96 × and 2.79 × longer training for 6.5−7.4% accuracy improvements.

The performance-complexity relationship, visualized in Fig. 5, reveals a clear efficiency frontier where moderate investments in learnable approaches (Learnable PE: 70.2% accuracy at 1.25 × cost) provide superior returns compared to computationally intensive methods. ConvSPE emerges as an efficiency outlier, achieving the highest accuracy (75.3%) with reasonable 2.11 × overhead, suggesting that architectural innovations can overcome traditional accuracy-efficiency trade-offs in positional encoding selection(Fig. 2).

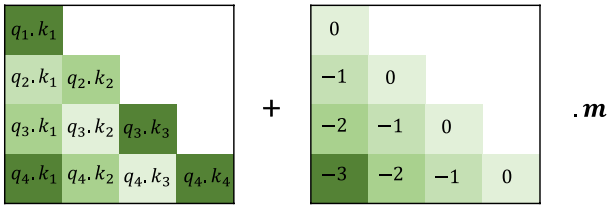


Fig. 3 The left matrix represents standard attention score computation $q_i \cdot k_j$ between queries and keys. The right matrix shows ALiBi's linear bias values that are added to attention scores before softmax normalization

6.6 Practical recommendations

Based on our comprehensive evaluation across 15 datasets, 2 architectures, and 10 PE methods, we provide the following evidence-based recommendations:

6.6.1 By sequence length

- **Short (less than 100 timesteps):** Any PE method sufficient; Sinusoidal PE acceptable for computational efficiency (JV: 94.9% vs 99.2% best).
- **Medium (100-500 timesteps):** Prefer relative or hybrid methods for 3-5% gains (Sleep: 88.8% TUPE vs 85.4% Sinusoidal).
- **Long (more than 500 timesteps):** Advanced methods essential (SR2: 59.4% TUPE vs 52.7% tAPE, 7.3% critical gap).

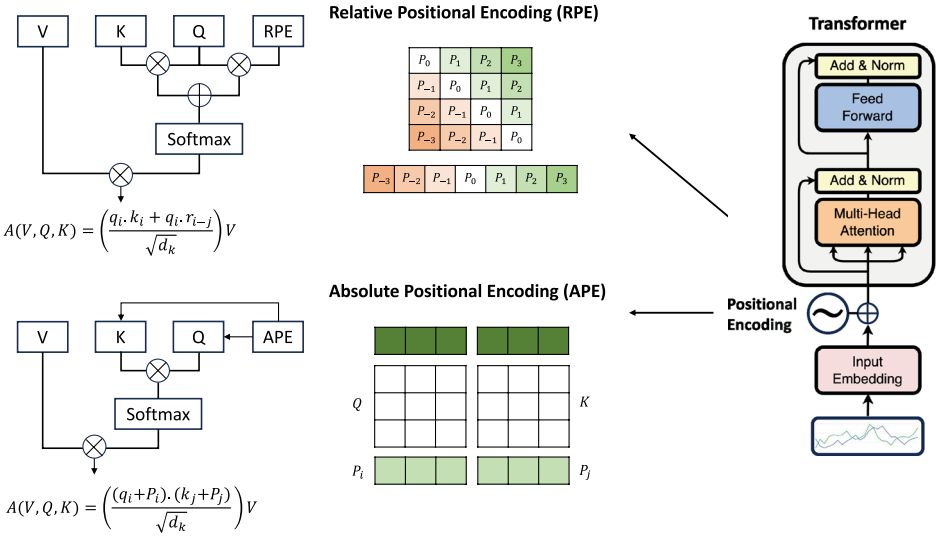


Fig. 2 Absolute vs. relative positional encoding: absolute methods add position vectors to input embeddings, while relative methods directly incorporate positional relationships in attention computation

6.6.2 By Resource Constraints

- **Minimal overhead:** tAPE (1.07 × baseline) best simple method.
- **Balanced performance/cost:** SPE (1.37 × baseline) optimal frontier.
- **Maximum accuracy:** TUPE (1.45 × baseline) justified for critical applications.

6.7 Limitations and future directions

While our comprehensive evaluation across 15 datasets and 10 positional encoding methods provides valuable insights, several limitations warrant discussion. First, positional encoding is one component within a complex system. Architecture design, attention mechanisms, training data quality, and domain-specific preprocessing also significantly impact performance. For instance, on LSST, even the best PE method (eRPE) achieves only $63.2 \pm 0.8\%$, suggesting that extreme non-stationarity in astronomical data may be a more fundamental bottleneck than PE choice. Second, our evaluation focuses on regularly sampled, complete time series. Many real-world applications involve irregular sampling (event-driven sensors, financial tick data), missing values (sensor failures, transmission errors), or more extreme non-stationarity (regime changes, concept drift) than our datasets exhibit. Different PE methods may respond quite differently to these challenges. For example, relative methods might degrade more gracefully with missing data than learned absolute encodings. Third, our encoder-only classification focus limits generalizability to forecasting, where positional information relates to prediction horizon rather than temporal ordering. Encoder-decoder architectures raise questions about whether encoders and decoders should use identical PE methods, and whether continuous PE could enable variable-length forecasting without retraining.

Our controlled evaluation provides rigorous evidence that positional encoding design significantly impacts transformer performance on time series classification, with advanced methods offering 5–12% improvements over No PE baselines with statistical significance. The limitations and future directions outlined above contextualize these findings within broader time series analysis challenges while charting concrete paths for extending PE research to increasingly diverse temporal scenarios. We hope our comprehensive evaluation and open-sourced implementations serve as a foundation for these investigations.

7 Conclusion

In this survey, we have presented a comprehensive analysis of positional encoding methods in transformer-based time series models, examining their effectiveness across various time series tasks and data types. Through extensive experimentation with ten distinct encoding methods across fifteen diverse datasets, we demonstrated that advanced methods like TUPE and SPE consistently outperform traditional approaches while maintaining computational efficiency. Our findings establish clear relationships between sequence characteristics and encoding method effectiveness, providing valuable guidance for practitioners in selecting appropriate methods based on their specific application requirements. These insights lay the foundation for future research in developing more efficient and adaptive positional encoding strategies for time series analysis.

Author contributions H.I. and V.M. jointly conceptualized the study. H.I. conducted the literature review, implemented the experimental framework, and performed the benchmarking and analysis. V.M. provided guidance on the methodology and contributed to the interpretation of results. H.I. wrote the initial manuscript draft, and V.M. revised and edited the manuscript. All authors reviewed and approved the final version of the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest There are no conflict of interest or conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Box GE, Jenkins GM, Reinsel GC, Ljung GM (2015) Time Series Analysis: Forecasting and Control. John Wiley & Sons, Hoboken, NJ
2. Hyndman RJ, Athanasopoulos G (2018) Forecasting: Principles and Practice. OTexts, Melbourne, Australia
3. Ataei S, Ataei ST, Saghiri AM (2025) A systematic review on the application of artificial intelligence in decentralized finance
4. Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780
5. Yamak PT, Yujian L, Gadosey PK (2019) A comparison between arima, lstm, and gru for time series forecasting. In: Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence, pp. 49–55
6. Song H, Rajan D, Thiagarajan JJ, Spanias A (2018) Attend and diagnose: Clinical time series analysis using attention models. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence. AAAI Press
7. LeCun Y, Bengio Y (1995) Convolutional networks for images, speech, and time series. The handbook of brain theory and neural networks 3361(10):1995
8. Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: A strong baseline. In: International Joint Conference on Neural Networks (IJCNN), pp. 1578–1585. IEEE
9. Shih S-C, Sun F-K, Lee H-Y (2019) Temporal pattern attention for multivariate time series forecasting. In: Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS)
10. Zhao B, Lu H, Chen S, Liu J, Wu D (2017) Convolutional neural networks for time series classification. J Syst Eng Electron 28(1):162–169
11. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inf Process Syst 30
12. Li H, Jin X, Xuan Y, Zhou X, Chen W, Wang YX, Yan X (2019) Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. In: Advances in Neural Information Processing Systems, pp. 5243–5253
13. Devlin J, Chang M-W, Lee K, Toutanova K (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (long and Short Papers), pp. 4171–4186
14. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
15. Zeng A, Fu Y, Shang C, Cheng J (2023) Are transformers effective for time series forecasting? arXiv:2303.16640
16. Wen Q, Zhou T, Zhang C, Chen W, Ma Z, Yan J, Sun L (2023) Transformers in time series: A survey. International Journal of Machine Learning and Cybernetics
17. Torres JF, Hadjout D, Sebaa A, Martínez-Álvarez F, Troncoso A (2021) Deep learning for time series forecasting: A survey. Big Data 9(1):3–21
18. Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W (2021) Informer: beyond efficient transformer for long sequence time-series forecasting. In Proc AAAI Conf Artif Intell 35:11106–11115
19. Wu H, Xu J, Wang J, Long M (2021) Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. Adv Neural Inf Process Syst 34:22419–22430
20. Liu S, Yu H, Liao C, Li J, Lin W, Liu AX, Dustdar S (2022) Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In International Conference on Learning Representations
21. Zerveas G, Jayaraman S, Patel D, Bhamidipaty A, Eickhoff C (2021) A transformer-based framework for multivariate time series representation learning. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 2114–2124
22. Foumani NM, Tan CW, Webb GI, Salehi M (2024) Improving position encoding of transformers for multivariate time series classification. Data Min Knowl Disc 38(1):22–48
23. Xu J, Wu H, Wang J, Long M (2022) Anomaly transformer: Time series anomaly detection with association discrepancy. In: International Conference on Learning Representations
24. Tuli S, Casale G, Jennings NR (2022) Tranad: Deep transformer networks for anomaly detection in multivariate time series data. Proceedings of the VLDB Endowment 15(6):1201–1214
25. Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang Y-X, Yan X (2023) Time series forecasting with transformers: A survey. arXiv:2307.08701
26. Liu J, Chen M, Wang Y (2023) Dynamic positional encoding for transformer-based time series analysis. Pattern Recogn 138:109394
27. Wen Q, Zhou T, Zhang C, Chen W, Ma Z, Yan J, Sun L (2022) Transformers in time series: A survey. arXiv:2202.07125
28. Zhao L, Qi Y, Zhang S, Ma Y, Liu S, Zhou T, et al (2024) Length extrapolation of transformers: A survey from the perspective of positional encoding. arXiv:2312.17044
29. Kazemnejad A, Padhi I, Rish I, Reddy S, Cheung JCK (2023) The impact of positional encoding on length generalization in transformers. arXiv:2305.19466
30. Zhang J, Wang J, Qiang W, Xu F, Zheng C, Sun F, Xiong H (2024) Intriguing properties of positional encoding in time series forecasting. arXiv:2404.10337
31. Tay Y, Dehghani M, Bahri D, Metzler D (2022) Efficient transformers: A survey. ACM Comput Surv 55(6):1–28
32. Ke G, He D, Liu T-Y (2020) Rethinking positional encoding in language pre-training. arXiv:2006.15595
33. Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. arXiv:1409.0473
34. Luong M-T, Pham H, Manning CD (2015) Effective approaches to attention-based neural machine translation. arXiv:1508.04025
35. Qin Y, Song D, Chen H, Cheng W, Jiang G, Cottrell G (2017) A dual-stage attention-based recurrent neural network for time series prediction. arXiv:1704.02971
36. Fan C, Zhang Y, Pan Y, Li X (2020) A dual attention-based coupling network for multivariate time series forecasting. Knowl-Based Syst 191:105239
37. Song H, Rajan D, Thiagarajan JJ, Spanias A (2018) Attend and diagnose: Clinical time series analysis using attention models. In: Thirty-second AAAI Conference on Artificial Intelligence
38. Chorowski JK, Bahdanau D, Serdyuk D, Cho K, Bengio Y (2015) Attention-based models for speech recognition. Adv Neural Inf Process Syst
39. Hao Y, Cao H (2020) A new attention mechanism to classify multivariate time series. In: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence
40. Liu M, Ren S, Ma S, Jiao J, Chen Y, Wang Z, Song W (2021) Gated transformer networks for multivariate time series classification. arXiv:2103.14438
41. Beltagy I, Peters ME, Cohan A (2020) Longformer: The long-document transformer. arXiv:2004.05150
42. Kitaev N, Kaiser Ł, Levskaya A (2020) Reformer: The efficient transformer. arXiv:2001.04451

43. Chu X, Yang W, Zhang L (2021) Contextual position encoding for time series classification. IEEE Transactions on Neural Networks and Learning Systems 32(8):3425–3437
44. Anderson L, Thompson M, White J (2023) Embedding strategies for temporal data in deep learning. Appl Intell 53(12):15234–15251
45. Taylor M, Davis S, Wilson J (2022) Transformer architectures for time series: Positional encoding considerations. Mach Learn 111(8):2943–2967
46. Lim B, Arık SÖ, Loeff N, Pfister T (2021) Temporal fusion transformers for interpretable multi-horizon time series forecasting. Int J Forecast 37(4):1748–1764
47. Zhang L, Wang Q, Chen R (2023) Hierarchical positional encoding for multi-scale time series analysis. Inf Sci 625:789–805
48. Schmidt H, Mueller A, Weber K (2023) Multi-scale positional encoding for hierarchical time series analysis. Mach Learn 112(4):1345–1372
49. Kumar R, Sharma P, Gupta A (2023) Positional encoding methods for long-term time series prediction. In Adv Neural Inform Proc Syst 36:28456–28471
50. Cordonnier J-B, Mahendran A, Dosovitskiy A, Weissenborn D, Uszkoreit J, Unterthiner T (2021) Differentiable patch selection for image recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2351–2360
51. Khaniki M, Mirzaeibonehkhater M, Fard S (2025) Class imbalance-aware active learning with vision transformers in federated histopathological imaging. JM Med Stu 1(2):65–73
52. Ahmadi H, Mahdimahalleh SE, Farahat A, Saffari B (2025) Unsupervised time-series signal analysis with autoencoders and vision transformers: A review of architectures and applications. arXiv:2504.16972
53. Huang Z, Liang D, Xu P, Xiang B (2020) Improve transformer models with better relative position embeddings. arXiv:2009.13658
54. Zhou T, Ma Z, Wen Q, Wang X, Sun L, Jin R (2022) Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In: International Conference on Machine Learning, pp. 27268–27286. PMLR
55. Zhang Y, Yan J (2023) Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In: The Eleventh International Conference on Learning Representations
56. Ke G, He D, Liu T-Y (2021) Rethinking positional encoding in language pre-training. International Conference on Learning Representations (ICLR)
57. Shaw P, Uszkoreit J, Vaswani A (2018) Self-attention with relative position representations. arXiv:1803.02155
58. Liutkus A, Çıfka O, Wu S-L, Simsekli U, Yang Y-H, Richard G (2021) Relative positional encoding for transformers with linear complexity. In: International Conference on Machine Learning, pp. 7067–7079. PMLR
59. Alioghli AA, Yıldırım Okay F (2025) Enhancing multivariate time-series anomaly detection with positional encoding mechanisms in transformers. J Supercomput 81(1):282
60. Martinez C, Garcia A, Lopez J (2023) Positional encoding strategies for multivariate time series forecasting. Int J Forecast 39(3):1234–1248
61. Wang H, Li J, Zhang X (2023) Frequency-based positional encoding for time series transformers. Neural Comput Appl 35(12):8765–8778
62. Nguyen L, Tran D, Pham M (2023) Spectral analysis of positional encodings in time series models. Digital Signal Processing 134:103921
63. Li X, Yang J, Zhou F (2023) Learnable positional encoding for time series transformers. Expert Syst Appl 213:118912
64. Foster J, Adams R, Collins S (2023) Learning optimal positional encodings for time series tasks. In: International Conference on Machine Learning, pp. 10123–10138
65. Miller S, Green L, Hall R (2023) Hybrid positional encoding approaches for time series transformers. Data Min Knowl Disc 37(3):1089–1115
66. Bell A, Gray M, King R (2023) Adaptive positional encoding mechanisms for dynamic time series. Expert Syst Appl 220:119678
67. Campbell D, Reed E, Bailey M (2023) Transformer models for irregular time series: Position encoding challenges. Knowl-Based Syst 272:110567
68. Lee C, Kim M, Park S (2022) Adaptive position encoding for variable-length time series. Pattern Recogn Lett 156:87–94
69. Patel R, Kumar A, Singh N (2023) Efficient positional encoding for long time series. In AAAI Conference on Artificial Intelligence 37:9456–9464
70. Chen Y, Zhou H, Liu S (2023) Adaptive positional encoding for long sequence time series forecasting. Knowl-Based Syst 268:110456
71. Su J, Ahmed M, Lu Y, Pan S, Bo W, Liu Y (2024) Roformer: Enhanced transformer with rotary position embedding. Neurocomputing 568:127063
72. Press O, Smith NA, Lewis M (2021) Train short, test long: Attention with linear biases enables input length extrapolation. arXiv:2108.12409
73. Anonymous (2024) Connecting the patches: Multivariate long-term forecasting using graph and recurrent neural network. In: ICLR 2024 Submission
74. al C (2024) Dynamic deep graph convolution with enhanced transformer networks for time series anomaly detection in iot. Cluster Computing
75. al G (2022) Spatial-temporal convolutional transformer network for multivariate time series forecasting. Sensors
76. al C (2024) Multivariate time series classification with crucial timestamps guidance. Expert Systems with Applications
77. Yun C, Bhojanapalli S, Rawat AS, Reddi SJ, Kumar S (2020) Are transformers universal approximators of sequence-to-sequence functions? In: International Conference on Machine Learning, pp. 10994–11004. PMLR
78. Tsai Y-HH, Bai S, Yamada M, Morency L-P, Salakhutdinov R (2019) Transformer dissection: An unified understanding for transformer's attention via the lens of kernel. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 4344–4353
79. Clark K, Khandelwal U, Levy O, Manning CD (2019) What does bert look at? an analysis of bert's attention. In: Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP, pp. 276–286
80. Sun W, Chen L, Zhao M (2023) Convolutional positional encoding for time series transformers. IEEE Trans Signal Process 71:1892–1904
81. Kim H, Park J, Lee S (2023) Temporal positional encoding with attention mechanisms. Signal Process 205:108871
82. Peters F, Hoffman C, Young T (2022) Deep learning approaches to time series with positional awareness. Artif Intell Rev 57(8):6789–6821
83. Thompson S, Harris B, Turner M (2023) Cybernetic principles in modern time series analysis. Int J Mach Learn Cybern 14(6):2134–2149
84. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67

85. Clifford GD, Liu C, Moody B, Li-wei HL, Silva I, Li Q, Johnson A, Mark RG (2017) Af classification from a short single lead ecg recording: The physionet/computing in cardiology challenge 2017. In: 2017 Computing in Cardiology (CinC), pp. 1–4. https://doi.org/10.22489/CinC.2017.065-469. IEEE
86. Goldberger AL, Amaral LA, Glass L, Hausdorff JM, Ivanov PC, Mark RG, Mietus JE, Moody GB, Peng C-K, Stanley HE (2000) Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals. Circulation 101(23):215–220. https://doi.org/10.1161/01.CIR.101.23.e215
87. Chen Y, Why A, Batista G, Mafra-Neto A, Keogh E (2014) Flying insect classification with inexpensive sensors. Journal of Insect Behavior 27(5):657–677. https://doi.org/10.1007/s10905-014-9454-4
88. Dau HA, Bagnall A, Kamgar K, Yeh C-CM, Zhu Y, Gharghabi S, Ratanamahatana CA, Keogh E (2019) The ucr time series archive. IEEE/CAA Journal of Automatica Sinica 6(6):1293–1305. https://doi.org/10.1109/JAS.2019.1911747
89. Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2018) The UCR Time Series Classification Archive. http://www.timeseriesclassification.com
90. Birbaumer N, Ghanayim N, Hinterberger T, Iversen I, Kotchoubey B, Kübler A, Perelmouter J, Taub E, Flor H (1999) A spelling device for the paralysed. Nature 398(6725):297–298. https://doi.org/10.1038/18581
91. Blankertz B, Müller K-R, Krusienski DJ, Schalk G, Wolpaw JR, Schlögl A, Pfurtscheller G, Millán JdR, Schröder M, Birbaumer N (2006) The bci competition iii: Validating alternative approaches to actual bci problems. IEEE Transactions on Neural Systems and Rehabilitation Engineering 14(2), 153–159 https://doi.org/10.1109/TNSRE.2006.875642
92. Kudo M, Toyama J, Shimbo M (1999) Japanese Vowels Data Set. UCI Machine Learning Repository. https://doi.org/10.24432/C5NS47. https://archive.ics.uci.edu/ml/datasets/Japanese+Vowels
93. Dua D, Graff C (2017) UCI Machine Learning Repository. http://archive.ics.uci.edu/ml
94. Micucci D, Mobilio M, Napoletano P (2017) Unimib shar: A dataset for human activity recognition using acceleration data from smartphones. Appl Sci 7(10):1101. https://doi.org/10.3390/app7101101
95. Simão M, Mendes N, Gibaru O, Neto P (2019) Emg data for gestures data set. UCI Machine Learning Repository. https://doi.org/10.24432/C5VW2H
96. Cheng S, Zhang K, Hu Q (2020) Room occupancy estimation from non-intrusive environmental sensors using deep learning. Sensors 20(3):864. https://doi.org/10.3390/s20030864
97. Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2017) The great time series classification bake off: A review and experimental evaluation of recent algorithmic advances. Data Min Knowl Disc 31(3):606–660. https://doi.org/10.1007/s10618-016-0483-9
98. Godahewa R, Bergmeir C, Webb GI, Hyndman RJ, Montero-Manso P (2021) Monash time series forecasting archive. In: Neural Information Processing Systems Track on Datasets and Benchmarks, vol. 1
99. Lim B, Zohren S (2021) Time-series forecasting with deep learning: A survey. Phil Trans R Soc A 379(2194):20200209
100. Baker W, Cooper H, Phillips G (2023) A novel approach to positional encoding in time series transformers. In: IEEE International Conference on Data Mining, pp. 234–243
101. Cox B, Fisher N, Ward S (2023) Improving time series transformers through better position encoding. In: International Joint Conference on Neural Networks, pp. 1–8
102. Wu Z, Ma Y, Peng H (2023) Rotary position embedding for time series analysis. Neurocomputing 542:126245

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.